



MEMORIA TÉCNICA

Sistema de Anonimización de Imágenes Médicas
mediante Inteligencia Artificial

Proyecto de Anonimización de Datos Médicos

Junio 2026

1. INTRODUCCIÓN Y OBJETIVOS

Este proyecto nace de la necesidad concreta de eliminar datos identificativos sobreimpresos en imágenes clínicas —nombres de paciente, ID, edad, fecha y hora— sin alterar la información médica relevante que contiene la radiografía. Hemos desarrollado una aplicación web que permite subir una imagen, procesarla localmente mediante una pipeline de inteligencia artificial y devolver la imagen limpia junto con un registro de todo lo que ha borrado.

El objetivo principal era construir una herramienta funcional, rápida y auto-contenida que no dependiera de servicios en la nube para el procesamiento, garantizando así que los datos médicos no salgan del servidor donde se despliega.

2. ARQUITECTURA DEL SISTEMA

La aplicación sigue una arquitectura clásica de tres capas: frontend, backend y motor de inferencia.

2.1. Frontend

La interfaz es una web estática (HTML, CSS y JavaScript) servida por Flask. El usuario arrastra o selecciona una imagen, pulsa *Anonimizar* y, mientras procesa, ve una animación de obturador que da feedback visual. Al terminar, aparece una comparación lado a lado (original vs anonimizada), el texto detectado y un botón de descarga.

2.2. Backend (Flask)

El servidor Flask gestiona tres rutas principales: **GET /** sirve la página principal; **GET /images/<archivo>** sirve los favicons; y **POST /anonimizar** recibe la imagen, la pasa al motor OCR y devuelve un JSON con la imagen anonimizada codificada en base64, las cajas detectadas normalizadas (0–1), el texto reconocido y metadatos del proceso.

Hemos optado por base64 en lugar de guardar archivos temporales en disco para evitar saturar el servidor con imágenes pesadas y simplificar el despliegue. Todo el procesado ocurre en memoria.

2.3. Pipeline de IA

El núcleo del sistema es una cadena de dos modelos de IA encadenados:

- **Detección:** un modelo YOLO entrenado a medida localiza las cajas de texto sobreimpreso en la imagen.
- **Reconocimiento:** PaddleOCR (PP-OCRv5) lee el contenido de cada caja detectada.
- **Borrado:** en lugar de tapar los rectángulos con parches de color, construimos una máscara fina de los trazos del texto y aplicamos *cv2.inpaint*, que reconstruye el fondo a partir de los píxeles vecinos. Así la imagen médica conserva su integridad visual.

2.4. Estructura de ficheros

- **app.py** — punto de entrada, servidor Flask.
- **ia/ocr_engine.py** — motor OCR reutilizable con carga perezosa de modelos.
- **ia/training.py** — script de entrenamiento del detector YOLO.
- **ia/conf/data.yaml** — configuración del dataset.
- **static/** — frontend (index.html, scripts.js, styles.css).

2.5. División del dataset y estrategia de evaluación

El dataset original que nos facilitaron venía dividido únicamente en dos conjuntos: *train* y *val*, con una proporción 80/20, sumando un total de 400 imágenes y sus 400 etiquetas correspondientes. Aunque esta división es válida para entrenar, no nos permitía evaluar de forma realista cómo se comportaba el modelo ante datos que no había visto nunca, así que decidimos reestructurarlo.

Redividimos el dataset en tres partes: **70 % train / 15 % val / 15 % test**, lo que nos dio **280 imágenes para entrenamiento, 60 para validación y 60 para test**. En *train* y *val* sí utilizamos las etiquetas: las de entrenamiento para ajustar los pesos del modelo YOLO y las de validación para monitorizar el rendimiento durante el entrenamiento y activar el *early stopping*. La diferencia está en el conjunto de *test*: **no utilizamos sus etiquetas en ningún momento del proceso**, ni para entrenar ni para validar. De este modo, las 60 imágenes de test actúan como un escenario completamente ciego, simulando una situación real en la que el detector se enfrenta a radiografías nuevas sin saber de antemano dónde está el texto.

Esta decisión nos permitió obtener métricas y resultados reales sobre qué tan bien detecta YOLO las cajas de texto en condiciones reales. El OCR (PaddleOCR PP-CRv5) se utiliza en este conjunto de test de forma directa, sin reentrenamiento, para evaluar su capacidad de lectura sobre las cajas que YOLO detecta en imágenes que el sistema no ha visto antes. Si hubiéramos mantenido solo train/val, estaríamos midiendo el rendimiento sobre datos que el modelo ya había visto de alguna forma durante el ajuste, lo que nos daría una falsa sensación de precisión.

3. LÓGICA DEL PLANTEAMIENTO

3.1. ¿Por qué YOLO para la detección?

Necesitábamos un detector rápido y preciso para objetos pequeños y alargados (como las líneas de texto sobreimpresas). YOLO, en su versión 12s, ofrece un buen equilibrio entre velocidad y precisión, y permite entrenar a medida con pocos datos si el data augmentation es inteligente.

3.2. ¿Por qué PaddleOCR para el reconocimiento?

PaddleOCR es robusto, soporta múltiples idiomas y su versión PP-OCRv5 tiene un rendimiento excelente en texto de baja calidad o fuentes no estándar —algo común en las sobreimpresiones de equipos médicos. Además, se integra bien con OpenCV y Python.

3.3. ¿Por qué inpainting en vez de rectángulos negros?

Tapar zonas con rectángulos de color destruye la información médica subyacente (por ejemplo, si el texto cae sobre una zona patológica). El inpainting (*cv2.inpaint*) reconstruye el fondo de forma heurística, preservando el contexto clínico de la imagen.

3.4. ¿Por qué Flask y no un framework más pesado?

Flask es ligero, modular y escrito en Python, lo que facilita la integración con PyTorch, OpenCV y PaddleOCR. Para una herramienta de uso interno o despliegue local, no necesitábamos la complejidad de Django o de un backend en Node.js.

3.5. Preprocesado múltiple

Las radiografías presentan condiciones extremas: texto sobre zonas *quemadas* (brillo irregular), texto blanco sobre fondo blanco casi invisible, o fuentes muy pequeñas. Por eso diseñamos un sistema de variantes de preprocesado que se aplican a cada recorte antes de pasarlo al OCR:

- **Variantes normales:** reescalado, CLAHE + nitidez, imagen invertida.
- **Variantes extremas:** estiramiento de contraste, CLAHE agresivo y binarización Otsu, para texto casi invisible.
- **Corrección de iluminación:** divide la imagen por un desenfoque grande del fondo para aplanar gradientes de brillo en zonas quemadas.

De todas las variantes, nos quedamos con la que devuelve mayor confianza del OCR.

4. RESOLUCIÓN DE DIFICULTADES TÉCNICAS

4.1. Texto sobre fondo de brillo irregular

En radiografías con zonas sobreexpuestas, el texto se pierde en el brillo del fondo. Implementamos *correccion_iluminacion*: aplicamos un desenfoque gaussiano grande para estimar el fondo y dividimos la imagen original por ese fondo, aplanando el gradiente y rescatando los trazos del texto.

4.2. Texto casi invisible (blanco sobre blanco)

Algunas sobreimpresiones usan texto blanco sobre fondos claros. Las variantes extremas con estiramiento de contraste adaptativo y binarización Otsu logran rescatar esos caracteres incluso cuando el ojo humano apenas los distingue.

4.3. Rendimiento y tiempos de carga

Los modelos de IA tardan varios segundos en cargar. Para que el servidor Flask arranque rápido y no falle si los pesos aún no están disponibles, implementamos carga perezosa (*lazy loading*): YOLO y PaddleOCR se cargan la primera vez que se invoca el endpoint, no al importar el módulo. Esto permite que la app esté operativa inmediatamente y de mensajes de error claros si falta algún modelo.

4.4. Tamaño de las imágenes y saturación del servidor

Las radiografías en alta resolución pueden pesar varios megabytes. En lugar de escribir archivos temporales en disco para cada petición, procesamos todo en memoria y devolvemos la imagen anonimizada como base64 dentro del JSON. Esto evita acumular ficheros temporales y reduce la latencia de escritura/lectura en disco.

4.5. Data augmentation para texto

El data augmentation estándar de YOLO (flip horizontal, rotaciones grandes) destruye la legibilidad del texto: una p volteada no es una q . Ajustamos los parámetros de augmentación: *flipplr=0*, *flipud=0*, rotación muy limitada y poca variación de tono. Así el modelo aprende a detectar texto en distintas posiciones sin distorsionar los caracteres.

4.6. Compatibilidad CPU vs GPU

PaddlePaddle tiene builds separadas para CPU y GPU. En *requirements.txt* incluimos la versión CPU por defecto para que funcione en cualquier máquina, pero dejamos documentado cómo cambiar a *paddlepaddle-gpu* si el usuario tiene CUDA disponible. Esto evita errores de instalación en entornos heterogéneos.

5. HERRAMIENTAS DE IA UTILIZADAS COMO APOYO

Durante el desarrollo de este proyecto hemos utilizado asistentes de inteligencia artificial (Kimi, Claude) como apoyo en las siguientes tareas:

- **Generación y depuración de código:** nos ayudaron a escribir el motor OCR, a resolver errores de instalación de PaddlePaddle-GPU y a optimizar el preprocesado de imágenes.
- **Estructuración de la documentación:** redacción de esta memoria técnica, el README del repositorio y guías de uso.
- **Diseño de la arquitectura:** sugerencias sobre la pipeline de detección + reconocimiento + inpainting, y decisiones sobre el formato de salida (base64).

Es importante aclarar que estos asistentes han sido herramientas de apoyo; la implementación, el entrenamiento del modelo YOLO, la toma de decisiones técnicas y el despliegue final han sido realizados por el equipo de desarrollo.

6. CONCLUSIONES

Hemos construido una herramienta funcional que anonimiza imágenes médicas mediante una pipeline de IA compuesta por detección (YOLO), reconocimiento (PaddleOCR) y reconstrucción del fondo (inpainting). El sistema es rápido, operable en CPU y preserva la integridad clínica de las radiografías. La aplicación web facilita su uso por personal no técnico, y la arquitectura modular permite reemplazar componentes (por ejemplo, cambiar el detector o el OCR) sin reescribir toda la aplicación.